

Projektdokumentation

ORBIT

Optimized Retrieval for Beyond Interstellar Technologies

Modul: Information Retrieval

Hochschule: Hochschule der Medien, Stuttgart

Semester: SS 2025

Dozent: Sascha Szott

Student: Patryk Gadziomski

Matrikelnummer: 43764

Abgabedatum: 27.07.2025

Inhaltsverzeichnis

1. ORBIT 3

2. Daten 4

3. Solr-Indexschema 9

4. Indexierungspipeline 10

5. Suchanfragetypen 11

6. Zusätzliche Funktionen 12

7. Web-App 13

8. Evaluation 14

9. Hinweise zur Hilfsmittel/KI-Nutzung 18

Literaturverzeichnis 19

Anhang 20

1. ORBIT

ORBIT (Optimized Retrieval for Beyond Interstellar Technologies) ist ein Such- und Wiederfindungssystem, das auf Apache Solr basiert. Die technische Architektur von ORBIT basiert auf einer Datenakquise- und Indexierungspipeline in Python sowie einer Webanwendung auf Basis von Flask. Die gesammelten und vorverarbeiteten Daten werden automatisiert in Apache Solr indiziert und über das Webinterface durchsuchbar gemacht. Es wurde entwickelt, um fortschrittliche Indexierungs- und Suchanfragen zu unterstützen, speziell für den Bereich intelligenter Systeme in der Raumfahrt und Erdbeobachtung.

Das System adressiert Themen wie „Künstliche Intelligenz in der Raumfahrt/Erdbeobachtung“, „Robotik in der Raumfahrt/Erdbeobachtung“ sowie alle weiteren Anwendungsfelder, in denen intelligente Systeme in dieser Domäne eingesetzt werden. Ziel ist es, wissenschaftliche Arbeitsabläufe zu unterstützen und das Auffinden relevanter Artikel in dieser datenintensiven Disziplin zu erleichtern.

ORBIT bietet eine einfache, aber moderne Benutzeroberfläche und verwendet Datenquellen wie arXiv und OpenAlex. Es werden verschiedene Suchanfragetypen unterstützt. Darüber hinaus helfen Funktionen wie Rechtschreibkorrektur und Ähnlichkeitsmetriken den Nutzenden dabei, passende Artikel auch bei ungenauen Suchanfragen zu finden.

2. Daten

Die für dieses Projekt verwendeten Daten stammen aus zwei Quellen: arXiv.org und OpenAlex.org. Alle genutzten Quellen unterliegen einer Creative-Commons-Lizenz. Allerdings sind einige Artikel im Datensatz nicht Open Access und unterliegen somit anderen Lizenzbedingungen. Solche Artikel werden bereits im Vorverarbeitungsschritt entfernt.

ArXiv ist ein frei zugänglicher Preprint-Server und Open-Access-Archiv für rund 2.4 Millionen wissenschaftliche Artikel aus den Fachbereichen Physik, Mathematik, Informatik, Biologie, Finanzwissenschaften, Statistik, Elektrotechnik und Wirtschaft. [1] OpenAlex ist ein umfassender wissenschaftlicher Katalog, der verschiedene Arten wissenschaftlicher Arbeiten wie Artikel, Datensätze, Bücher und Dissertationen erfasst. Dabei werden nicht nur Titel, Zusammenfassungen und Volltexte gespeichert, sondern auch strukturelle Verbindungen zwischen wissenschaftlichen Arbeiten. Dazu zählen Assoziationen zwischen Zeitschriften, Autoren, Institutionen, Zitationen, Konzepten und Förderorganisationen. [2]

In ORBIT wird arXiv verwendet, um die wissenschaftlichen Artikel selbst zu beziehen. OpenAlex dient dazu, diese Artikel mit zusätzlichen Metadaten anzureichern.

Datenformat: JSONL

arXiv											
Arxiv ID	DOI	Titel	Autor	Abstrakt	Veröf. Datum	Akt. Datum	Journal	Kategorie	Link	PDF-Link	Volltext
Str	Str	Str	List(Str)	Str	Str	Str	Str	List(Str)	Str	Str	Str

Datenformat: JSONL

OpenAlex									
OpenAlex ID	Themen	Anzahl Zitierungen	Veröf. Ort	Open Access	Typ	Sprache	Lizenz	Paratext	Autor
Str	List(dict)	Int	List(dict)	Boolean	Str	Str	Str	boolean	List(dict)

Zitierte Artikel	Verwandte Artikel	Biblio	Institutionen	Förderungen	Konkrete Fundstelle	Keywords
List(Str)	List(Str)	dict	List(dict)	List(dict)	dict	List(dict)

Abbildung 1: ArXiv und OpenAlex Daten nach der Akquisition

Die arXiv-Daten (vgl. Abbildung 1) bestehen aus zwölf Spalten. Neben den wichtigsten Angaben wie *ID*, *DOI*, *Autoren*, *Erscheinungsdatum* und *Abstract* enthalten sie auch Informationen zum *Journal*, zur *Kategorie* sowie Links zum *Artikel* und zum zugehörigen *PDF*. Der Volltext der Artikel wird in einem nachgelagerten Schritt mithilfe eines Python-Skripts und der Bibliothek *fitz* aus den PDF-Dateien extrahiert und anschließend bereinigt. OpenAlex liefert darüber hinaus eine Vielzahl zusätzlicher Metadaten (vgl. Abbildung 1). Für die Indexierung werden vor allem folgende Felder

verwendet: *OpenAlex-ID*, *Themen*, *Anzahl der Zitierungen*, *Dokumenttyp*, *Sprache*, *Lizenz*, *Autoren*, *Zitierte Artikel*, *Verwandte Artikel*, *Bibliografische Angaben* (z. B. Ausgabe, Seitenzahlen), *Institution*, *Stichwörter*.

Andere Spalten wurden verworfen, entweder aufgrund fehlender Relevanz für den Anwendungsfall oder wegen eines zu hohen Anteils fehlender Werte. Die Datentypen der einzelnen Felder sind Abbildung 1 zu entnehmen. In diesem Projekt liegt eine Kombination unterschiedlich strukturierter Daten vor: Einerseits strukturierte Daten wie *Autoren*, *Themen* oder *Stichwörter*, die im JSON-Format vorliegen, andererseits unstrukturierte Daten wie der *Abstract* und der *Volltext*, die als reiner Fließtext gespeichert sind.

Die Datenakquise erfolgte über ein Python-Skript, das die APIs der jeweiligen Datenquellen (arXiv und OpenAlex) aufruft und die Daten automatisiert abfragt. Die Daten von arXiv werden mithilfe einer vordefinierten Query abgefragt, um sicherzustellen, dass sie thematisch zum Use Case passen. Dabei werden verschiedene Schlüsselwortkombinationen im Titel und Abstract getestet, um eine möglichst große Menge relevanter Publikationen zu erfassen. Die abgerufenen arXiv-Daten werden im JSONL-Format gespeichert und anschließend mit Metadaten aus OpenAlex angereichert. Der Match zwischen den Datensätzen erfolgt über die *DOI*, *arXiv-ID* oder den *Artikeltitel*, je nachdem, welche Informationen jeweils verfügbar sind. Das JSONL-Format wurde gewählt, da es dem gängigen JSON-Format entspricht und strukturierte (verschachtelte) Speicherung erlaubt, jedoch deutlich speichereffizienter ist. Dies ist besonders vorteilhaft bei großen Datenmengen. Durch die Anreicherung entsteht ein kombinierter Datensatz mit zunächst ca. 1.200 Artikeln. Da jedoch nicht alle Artikel erfolgreich zwischen arXiv und OpenAlex gematcht werden konnten, reduziert sich der finale Datensatz auf 1.163 Einträge.

```
query = '((ti:"artificial intelligence" OR ti:"machine learning" OR
ti:"deep learning" OR ti:"computer vision" OR ti:"natural language
processing" OR ti:"robotics" OR ti:"autonomous systems" OR
ti:"automation") OR (abs:"artificial intelligence" OR abs:"machine
learning" OR abs:"deep learning" OR abs:"computer vision" OR
abs:"natural language processing" OR abs:"robotics" OR
abs:"autonomous systems" OR abs:"automation")) AND ((ti:satellite OR
ti:spacecraft OR ti:aerospace OR ti:"remote sensing" OR ti:"earth
observation" OR ti:"space exploration" OR ti:"planetary science") OR
(abs:satellite OR abs:spacecraft OR abs:aerospace OR abs:"remote
sensing" OR abs:"earth observation" OR abs:"space exploration" OR
abs:"planetary science"))'
```

Eine erste Analyse des Datensatzes zeigte deutliche Merkmale, die auf weitere notwendige Vorverarbeitungsschritte hinwiesen. Zunächst wurde die Häufigkeit der *Concepts* (Themen) der Artikel untersucht, um festzustellen, ob der Großteil der Dokumente thematisch zum Use Case passt. Wie in Anhang 1 ersichtlich, sind die beiden häufigsten Themen „Computer Science“ und „Artificial Intelligence“, was im Kontext der Domäne „Intelligente Systeme“ durchaus erwartbar und relevant ist. Auch die weiteren häufigen Konzepte beziehen sich auf „Physik“, „Raumfahrt“, „Erdbeobachtung“ oder technisch-wissenschaftliche Disziplinen, was den thematischen Fokus zusätzlich bestätigt. Diese Analyse zeigt, dass die verwendete Query erfolgreich war und ein sinnvoll gefilterter arXiv-Datensatz erzeugt wurde. Ein weiterer wichtiger Aspekt war die Überprüfung des Open-Access-Anteils im Datensatz (vgl. Anhang 2). Von den ursprünglich 1.163 Artikeln waren 84 nicht Open Access und wurden daher aus dem Datensatz entfernt. Der finale Datensatz umfasst somit 1.079 wissenschaftliche Artikel. Die Analyse der fehlenden Werte pro Spalte (vgl. Anhang 3) diente dazu, irrelevante oder unvollständige Felder zu identifizieren. Dabei zeigte sich, dass bestimmte Felder keine Informationen enthalten: *institutions*, *host_venue_license*, *host_venue*. Diese Spalten wurden daher aus dem Datensatz entfernt. Zusätzlich weisen die Felder *journal* und *doi* in über 50% der Einträge fehlende Werte auf. Abschließend wurde der zeitliche Verlauf der Veröffentlichungen im Zeitraum von 2000 bis 2023 untersucht (vgl. Anhang 4). Auffällig ist ein deutlicher Rückgang in den Jahren 2021 und 2022, was vermutlich auf die Auswirkungen der COVID-19-Pandemie zurückzuführen ist. Im Jahr 2023 zeigt sich hingegen eine vollständige Erholung der Veröffentlichungszahlen.

Nach der explorativen Datenanalyse folgt die Datenverarbeitung, bei der Spalten mit geringer Relevanz entfernt und andere gezielt vorverarbeitet wurden. Aus Gründen der geringen Aussagekraft bzw. hoher Anzahl fehlender Werte wurden folgende Spalten aus dem Datensatz entfernt:

- **link** (Information bereits durch arXiv-ID bzw. OpenAlex-ID vorhanden)
- **authors** (die Autoreninformationen von OpenAlex werden bevorzugt verwendet)
- **category** (ebenfalls durch OpenAlex-Konzepte ersetzt)
- **host_venue** (zu viele fehlende Werte)

- **host_venue_license** (zu viele fehlende Werte)
- **mesh** (fehlende Relevanz für den Use Case)
- **grants** (fehlende Relevanz)
- **is_paratext** (fehlende Relevanz)
- **institutions** (keine Werte im Datensatz vorhanden)

Der Volltext der Artikel wurde zusätzlich bereinigt, indem Tabellen und lange Zahlenreihen entfernt wurden, um eine bessere Weiterverarbeitung zu ermöglichen. Verschachtelte Felder wie *concepts*, *primary_location*, *keywords*, *authorships* und *biblio* wurden entpackt. Das bedeutet, dass aus den jeweiligen JSON-Strukturen nur die Informationen extrahiert wurden, die für die spätere Nutzung relevant sind. Die final bereinigten und transformierten Daten wurden anschließend in einer neuen JSONL-Datei gespeichert.

Für das Relevanzranking der Suchergebnisse wurden mehrere Merkmale der Artikel gewichtet, um die Qualität und Präzision der Treffer zu verbessern. Neben dem *Titel*, den *Autoren*, dem *Abstract* und dem *Volltext* spielen auch die *Konzepte* (Themen) eine zentrale Rolle. Die Gewichtung der Felder wurde wie folgt festgelegt:

- **Titel**: Gewicht × 5
- **Autoren**: Gewicht × 3
- **Abstract**: Gewicht × 2
- **Volltext**: Gewicht × 1
- **Konzepte (Concepts)**: Gewicht × 1

Zusätzlich wird das Ranking durch die Anzahl der Zitationen beeinflusst (Boost-Faktor: *log(cited_by_count × 2)*). Ziel dieser Gewichtung ist es, relevantere und qualitativ hochwertigere Artikel in den Suchergebnissen weiter oben anzuzeigen.

Potenzial für Optimierungen besteht insbesondere in der weiterführenden Vorverarbeitung der Volltexte. In diesem Projekt wurde diese bislang nur grundlegend umgesetzt. Eine tiefere Textaufbereitung könnte die Qualität der Indexierung und damit auch die Relevanz der Suchergebnisse deutlich verbessern. Auch im Bereich des Relevanzrankings besteht weiteres Entwicklungspotenzial. Es sollten verschiedene

Gewichtungskombinationen und Boosting-Strategien systematisch getestet werden, um eine möglichst optimale Relevanzbewertung zu erreichen.

3. Solr-Indexschema

```
FIELDS = [
    {"name": "arxiv_id", "type": "string", "stored": True, "indexed": True, "required": True},
    {"name": "openalex_id", "type": "string", "stored": True, "indexed": True, "required": True},
    {"name": "doi", "type": "string", "stored": True, "indexed": True, "required": False},
    {"name": "title", "type": "text_general", "stored": True, "indexed": True},
    {"name": "summary", "type": "text_general", "stored": True, "indexed": True, "multiValued": False},
    {"name": "published", "type": "date", "stored": True, "indexed": True, "docValues": True},
    {"name": "updated", "type": "date", "stored": True, "indexed": True, "docValues": True},
    {"name": "pdf_link", "type": "string", "stored": True, "indexed": True},
    {"name": "fulltext", "type": "text_custom", "stored": False, "indexed": True},
    {"name": "cited_by_count", "type": "pint", "stored": True, "indexed": False, "docValues": True},
    {"name": "type", "type": "string", "stored": True, "indexed": True, "docValues": True},
    {"name": "referenced_works", "type": "text_general", "stored": True, "indexed": True, "multiValued": True},
    {"name": "related_works", "type": "text_general", "stored": True, "indexed": True, "multiValued": True},
    {"name": "grants", "type": "string", "stored": True, "indexed": True, "docValues": True},
    {"name": "landing_page_url", "type": "string", "stored": True, "indexed": True},
    {"name": "author_orcid", "type": "string", "stored": True, "indexed": True, "multiValued": True},
    {"name": "volume", "type": "string", "stored": True, "indexed": True},
    {"name": "issue", "type": "string", "stored": True, "indexed": True},
    {"name": "first_page", "type": "string", "stored": True, "indexed": True},
    {"name": "last_page", "type": "string", "stored": True, "indexed": True},
    {"name": "journal", "type": "string", "stored": True, "indexed": True, "multiValued": "true", "docValues": True},
    {"name": "concepts", "type": "string", "stored": True, "indexed": True, "multiValued": True, "docValues": True},
    {"name": "author_names", "type": "text_general", "stored": True, "indexed": True, "multiValued": True, "docValues": True},
    {"name": "open_access", "type": "boolean", "stored": True, "indexed": False, "multiValued": False, "docValues": True},
    {"name": "language", "type": "string", "stored": True, "indexed": True, "multiValued": True,"docValues": True},
    {"name": "keywords", "type": "string", "stored": True, "indexed": True, "multiValued": True, "docValues": True},
    {"name": "spellcheck_base", "type": "text_general", "stored": False, "indexed": True, "multiValued": True}
]
```

Abbildung 2: Indexschema ORBIT

Das für ORBIT verwendete Indexschema definiert, wie die einzelnen Felder im Suchindex behandelt werden. Jedes Feld besitzt dabei drei zentrale Eigenschaften:

- **stored=true**: Das Feld wird im Index gespeichert und kann beim Abruf eines Dokuments angezeigt werden (z. B. in der Ergebnisliste oder Detailansicht).
- **indexed=true**: Das Feld wird für die Volltextsuche indexiert und steht für Suchanfragen zur Verfügung.
- **docValues=true**: Das Feld wird für Facettierung, Sortierung und Aggregation vorbereitet. Dies ist besonders relevant für analytische Auswertungen und Filterfunktionen im Frontend.

Zusätzliche Eigenschaften wie *required* oder *multiValued* werden gemäß den Anforderungen gesetzt und gelten bei vielen Feldern als selbstverständlich. Das verwendete Schema umfasst insgesamt 27 Felder, die sich auf sechs verschiedene Feldtypen verteilen (*text_general*, *string*, *boolean*, *pdate*, *plong*, *pfloat*). Die genaue Konfiguration dieser Felder orientiert sich an ihrer jeweiligen Funktion im Suchprozess, etwa ob sie durchsuchbar, filterbar oder ausschließlich zur Anzeige gedacht sind.

4. Indexierungspipeline

Die Indexierung der Daten erfolgt über das Python-Skript *index_main.py*. Die Konfiguration der solr cores, ports und hosts sind über die *config.yml* möglich. Diese wird über *orbit_utils/orbit_config_loader.py* geladen.

Dieses Skript durchläuft mehrere aufeinanderfolgende Schritte:

1. **Datensammlung:** Zunächst werden die Daten geladen, die für die Indexierung verwendet werden sollen. (Bibliotheken: *urllib*, *xml*, *json*, *requests, fitz*, *io;* Skript: *orbit_utils/data_access.py*)
2. **Datenbereinigung:** Vor dem Indexieren erfolgt eine finale Bereinigung und ggf. Umstrukturierung der Daten. (Bibliotheken: *pandas*, *re;* Skript: *orbit_utils/data_cleaning.py*)
3. **Indexschema-Generierung:** Das Indexschema wird automatisch generiert und es wird sichergestellt, dass alle Felder korrekt definiert sind (inkl. *stored*, *indexed*, *docValues*, etc.). (Bibliotheken: *requests*; Skript: *orbit_utils/create_solr_schema.py*)
4. **Indexierung:** Die Daten werden an Solr gesendet und gemäß Schema indexiert. Dabei wird sichergestellt, dass alle Pflichtfelder vorhanden sind und die Typen korrekt gesetzt wurden. (Bibiotheken: *requests*; Skript: *orbit_utils/index_orbit.py*)
5. **Spellcheck-Feld:** Zusätzlich wird ein spezielles Spellcheck-Feld erstellt und indexiert. Dieses basiert auf einer Kombination der Felder *title*, *summary* und *full_text* und dient dazu, Benutzereingaben bei Suchanfragen automatisch auf mögliche Tippfehler zu prüfen und entsprechende Korrekturvorschläge zu liefern.

5. Suchanfragetypen

ORBIT unterstützt unterschiedliche Suchanfragetypen, um den Nutzer flexible und präzise Recherchemöglichkeiten zu bieten. Zu den derzeit implementierten Funktionen gehören:

- **Keyword-Suche**: Die einfachste Form der Suche. Nutzer geben einen oder mehrere Begriffe ein, die im Titel, Abstract, Volltext oder anderen durchsuchbaren Feldern vorkommen können.
- **Wildcard-Suche**: Die Wildcard-Suche erlaubt die Verwendung von Platzhaltern wie * (beliebig viele Zeichen), um Begriffe teilweise zu maskieren. Platzhalter können am Anfang, Ende oder innerhalb eines Suchbegriffs stehen. **Beispiel**: *intelligen** findet sowohl *intelligent*, *intelligence* als auch *intelligente*.
- **Phrasensuche**: Durch das Setzen von Begriffen in Anführungszeichen ("") kann gezielt nach exakten Wortgruppen gesucht werden. **Beispiel**: *"*artificial intelligence*"* liefert nur Treffer mit exakt dieser Wortkombination.
- **Facettensuche**: Die Facettensuche erlaubt es Nutzern, ihre Ergebnisse mithilfe dynamischer Filter weiter einzugrenzen. In der aktuellen Version von ORBIT stehen folgende Filter zur Verfügung:
    - **Concept** (Thema)
    - **Open Access** (ja/nein)
    - **Sprache** des Artikels
    - **Zeitschrift** (Journal)
    - **Typ** (z. B. Artikel, Dissertation, Datensatz)

6. Zusätzliche Funktionen

ORBIT bietet neben den verschiedenen Suchanfragetypen auch Funktionen zur Verbesserung der Suchqualität, insbesondere bei fehlerhaften oder unvollständigen Eingaben.

Bei Suchanfragen mit Tipp- oder Rechtschreibfehlern erkennt ORBIT die Abweichung und schlägt korrekte Begriffe vor. Grundlage dafür ist die interne Berechnung der Damerau-Levenshtein-Distanz, die auch Transpositionen (z. B. „artifical“ statt „artificial“) berücksichtigt. Dadurch werden auch leicht fehlerhafte Eingaben sinnvoll interpretiert und alternative Vorschläge generiert. Dazu werden pro ausgewählten Artikel die Top 10 ähnlichsten Artikel vorgeschlagen. Dies soll dem Nutzer bei der Recherche helfen, um ähnliche Artikel effizienter entdecken zu können.

Einige weiterführende Funktionen konnten aus zeitlichen Gründen noch nicht umgesetzt, sind aber für zukünftige Versionen vorgesehen:

- **Autovervollständigung**: Vorschläge während der Eingabe von Suchbegriffen, basierend auf häufigen oder relevanten Termen im Index.
- **Highlighting**: Markierung der gesuchten Begriffe im Titel oder Abstract der Suchergebnisse zur besseren Übersicht.

7. Web-App

Die Web-App von ORBIT wurde mit Python, dem Flask-Framework, Jinja und JavaScript umgesetzt. Apache Solr läuft in einem Docker-Container. Das Frontend basiert auf HTML, CSS und Bootstrap. Grafische Elemente wie Icons und Illustrationen wurden mit Canva erstellt. Die Anwendung wurde unter Windows 11 mit den Browsern Google Chrome und Mozilla Firefox erfolgreich getestet.

In Abbildung 3 ist das umgesetzte Design sowie die Ergebnisansicht dargestellt. Die Benutzeroberfläche ist wie folgt aufgebaut:

- Oben links befindet sich die Suchleiste, ergänzt um auswählbare Facettenfilter.
- Darunter erscheint eine paginierte Ergebnisliste, die die relevantesten Treffer zur Suchanfrage anzeigt.
- Durch einen Klick auf „More“ öffnet sich im rechten Bereich eine Detailansicht mit den wichtigsten Informationen zum ausgewählten Artikel.
- Unterhalb dieser Detailansicht befindet sich die Sektion „Similar Documents“, in der die inhaltlich ähnlichsten Artikel gelistet werden, berechnet auf Basis von Textähnlichkeitsmetriken.

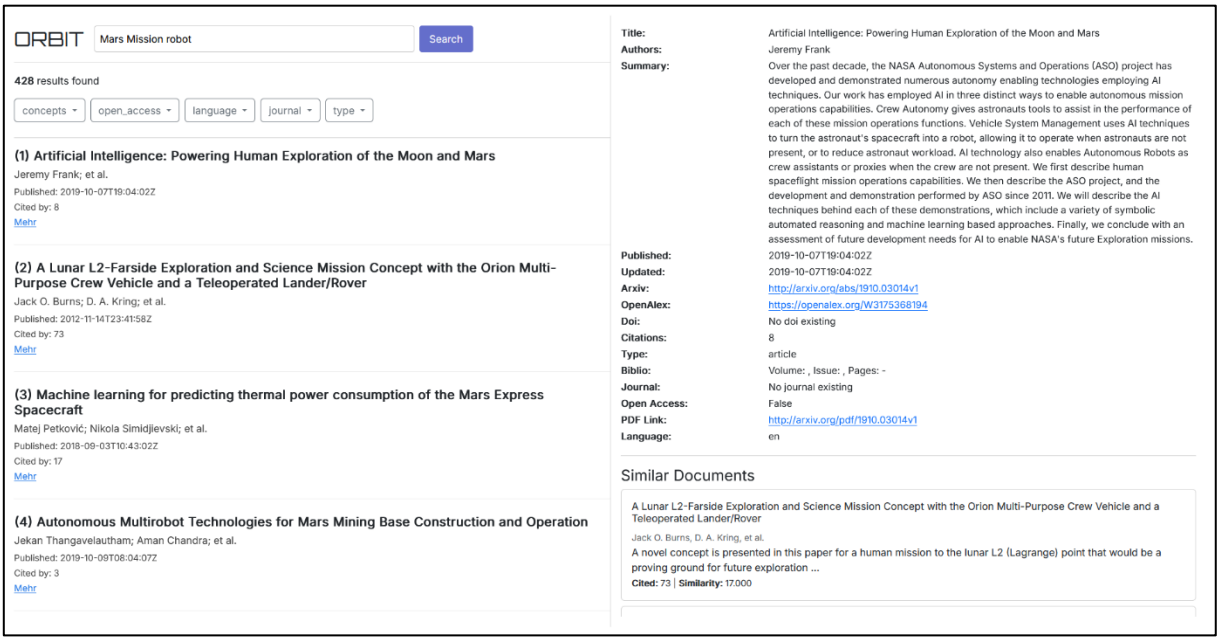


Abbildung 3: ORBIT Web-App

8. Evaluation

Da es kein einzelnes Maß gibt, das die Effektivität eines Information-Retrieval-Systems (IR-Systems) vollständig abbildet, empfiehlt es sich, mehrere Evaluationsmetriken in Kombination zu betrachten. Wie in den Vorlesungsfolien des Moduls „Information Retrieval“ vorgeschlagen, wird dazu eine Menge von Anfragen *Q* verwendet, um folgende Kennzahlen für ein IR-System zu berechnen:

- **Mean Average Precision (MAP)**, ggf. nur basierend auf den relevanten Dokumenten im Top-10-Ranking
- **Precision-Recall-Kurve**, ggf. beschränkt auf das Top-10-Ranking
- **Average Rank-10 Precision**

Mean Average Precision (MAP)

$$MAP(q_i) = \frac{1}{rel(q_i)} \sum_{j=1}^{rel(q_i)} Prec(R(q_i, j))$$

Die Mean Average Precision (MAP) liefert eine Kennzahl, die sich besonders gut zum Vergleich verschiedener IR-Systeme eignet. Für einzelne Testanfragen aus der Anfragemenge *Q* wird ein Parameter *k* festgelegt, anhand dessen die entsprechenden Metriken berechnet werden.

Zur Evaluation wurden fünf Themenbereiche definiert, die jeweils vier spezifische Suchanfragen enthalten. Für jeden Bereich wurde die MAP berechnet. Die Ergebnisse sind wie folgt:

- **satellite_imagery_analysis**: MAP = 0.078
    - satellite imagery analysis deep learning
    - satelite image segmentation convolutional neural network
    - deep learning approaches for analyzing multispectral satellite imagery for vegetation health monitoring
    - how to classify land cover change from Sentinel-2 images using CNNs
- **earth_observation**: MAP = 0.15
    - earth observation machine learning

    - EO satellite sensors atmospheric parameter retrieval
    - how can atmospheric parameters be reconstructed from earth observation data?
    - machine learning retrieval of soil moisture from microwave remote sensing data
- **natural_language_processing**: MAP = 0.00
    - NLP for scientific paper retrieval in space mission planning
    - transformer models summarizing satellite telemetry research articles
    - how can NLP be used to find relevant papers on robotics control in spacecraft?
    - using BERT to extract key findings from Earth observation research articles
- **robotics_in_spacecraft**: MAP = 0.13
    - robotics in spacecraft
    - AI robotics autonomous spacecraft control systems
    - how to optimize neural networks for autonomous spacecraft navigation?
    - deep reinforcement learning for satellite docking manoeuvres
- **remote_sensing**: MAP = 0.15
    - remote sensing hyperspectral image unmixing deep learning
    - SAR InSAR deformation detection earth observation
    - AI techniques for spectral unmixing of hyperspectral remote sensing data
    - how to detect urban land subsidence via InSAR for remote sensing analysis

Der durchschnittliche MAP-Wert über alle Themenbereiche beträgt 0.075. Wird das Thema natural_language_processing aufgrund seines MAP-Werts von 0.0 ausgeschlossen, ergibt sich ein durchschnittlicher MAP-Wert von 0.08.

Wie sich anhand der MAP-Werte erkennen lässt, erzielt keines der Themengebiete, auch nicht im Gesamtdurchschnitt, einen besonders hohen Wert. Dies liegt vor allem daran, dass relevante Dokumente entweder nur vereinzelt vorhanden sind oder im Ranking überwiegend weiter hinten erscheinen.

Precision-Recall-Kurve

Im Anhang sind die Precision-Recall-Kurven der jeweiligen Themencluster dargestellt, basierend auf einer Ranking-Länge von *k=10*. Dabei wird deutlich, dass die Kurven starke Schwankungen aufweisen, die Werte springen häufig von hoch zu niedrig und zurück. Dieses Verhalten lässt Rückschlüsse auf die Verteilung relevanter Dokumente im Ranking zu: In einigen Clustern befinden sich relevante Ergebnisse eher am Anfang, in anderen weiter hinten. Auch die Anzahl relevanter Dokumente variiert deutlich zwischen den Themen.

Average Precision (AP)

Zur besseren Übersicht der Ergebnisse wurde zusätzlich die Average Precision (AP) für das Ranking mit *k=10* berechnet (vgl. Tabelle 1). Diese Kennzahl wird sowohl für jedes einzelne Query innerhalb eines Clusters als auch aggregiert pro Cluster und insgesamt ermittelt. Aus der entsprechenden Ergebnistabelle lässt sich ablesen, dass manche Suchanfragen deutlich bessere Resultate liefern als andere. Ebenso zeigt sich, dass bestimmte Cluster insgesamt leichter auffindbare Inhalte enthalten als andere. So erzielt beispielsweise der Cluster *natural_language_processing* (Cluster 3) kaum relevante Treffer. Dies deutet darauf hin, dass entsprechende Inhalte in der zugrunde liegenden Datenbasis des IR-Systems kaum oder gar nicht vorhanden sind.

Cluster	Query	AP per Query	AP per Cluster	AP for all queries
satellite_imagery	Query 1	**1.000**	0.847	**0.659**
	Query 2	**1.000**		
	Query 3	0.511		
	Query 4	0.878		
earth_observation	Query 1	**1.000**	0.816	
	Query 2	0.797		
	Query 3	0.583		
	Query 4	0.583		
natural_language_processing	Query 1	**0.292**	0.073	
	Query 2	0.000		
	Query 3	0.000		
	Query 4	0.000		
robotics_in_spacecraft	Query 1	0.909	0.706	
	Query 2	**0.928**		
	Query 3	0.307		
	Query 4	0.679		
remote_sensing	Query 1	0.698	**0.854**	
	Query 2	0.849		
	Query 3	**0.959**		
	Query 4	0.909		

Tabelle 1: Average Precision pro Query, Cluster und insgesamt

9. Hinweise zur Hilfsmittel/KI-Nutzung

Im Rahmen der Erstellung dieser Arbeit wurden verschiedene Hilfsmittel zur Unterstützung eingesetzt. Dazu zählten insbesondere:

- **Kursmaterialien**: Der bereitgestellte Beispielcode [3] des Dozenten sowie die Vorlesungsfolien [4] dienten als Grundlage zum Verständnis technischer Konzepte und zur praktischen Umsetzung.
- **KI-gestützte Werkzeuge**: Es kamen Sprachmodelle wie ChatGPT (OpenAI) [5] und Claude (Anthropic) [6] zum Einsatz. Diese wurden gezielt zur inhaltlichen Unterstützung und zur Verbesserung der Dokumentation genutzt.

Die Verwendung der Sprachmodelle erfolgte in folgenden konkreten Anwendungsfällen:

- **Evaluation der Suchmaschine**: Über ChatGPT wurde erfragt, wie die *Mean Average Precision* zur Bewertung von Suchergebnissen genutzt werden kann. Zudem wurde um ein Beispiel zur Automatisierung der Berechnung sowie eine Erklärung der einzelnen Schritte gebeten.
- **Fehlerbehebung bei der „Similar Documents“-Funktion**: Mit Claude wurde ein Fehler analysiert, bei dem beim Abrufen ähnlicher Dokumente ein 404-Fehler in der Konsole angezeigt wurde (u. a. URL-Encoding-Problem bei GET /mlt/…). Die Hinweise halfen bei der Eingrenzung der Ursache.
- **Stilistische Überarbeitung**: ChatGPT wurde verwendet, um Hinweise zur Verbesserung des wissenschaftlichen Schreibstils der Dokumentation zu erhalten. Dabei wurden gezielt Feedback zu Ausdruck, Satzbau und Formulierung erbeten, um den Text sprachlich aufzuwerten.
- **Automatisierungsidee (verworfen)**: Es wurde zudem überlegt, mit ChatGPT eine automatisierte Bearbeitung der solrconfig.xml-Datei für Apache Solr umzusetzen. Die Idee war, gezielt bestimmte XML-Bereiche programmatisch zu verändern. Nach Abwägung der Komplexität wurde diese Vorgehensweise jedoch nicht weiterverfolgt.

Alle KI-gestützten Anfragen wurden bewusst und reflektiert eingesetzt. Die endgültige inhaltliche Verantwortung sowie die Auswahl, Bewertung und Integration der Antworten lagen stets beim Autor dieser Arbeit.

Literaturverzeichnis

[1] „About arXiv - arXiv info,“ 26 07 2025. [Online]. Available: https://info.arxiv.org/about/index.html.

[2] „OpenAlex - About us,“ 26 07 2025. [Online]. Available: https://help.openalex.org/hc/en-us/articles/24396686889751-About-us.

[3] S. Szott, „github.com/saschaszott/ir-hdm-2025,“ 26 07 2025. [Online]. Available: https://github.com/saschaszott/ir-hdm-2025.

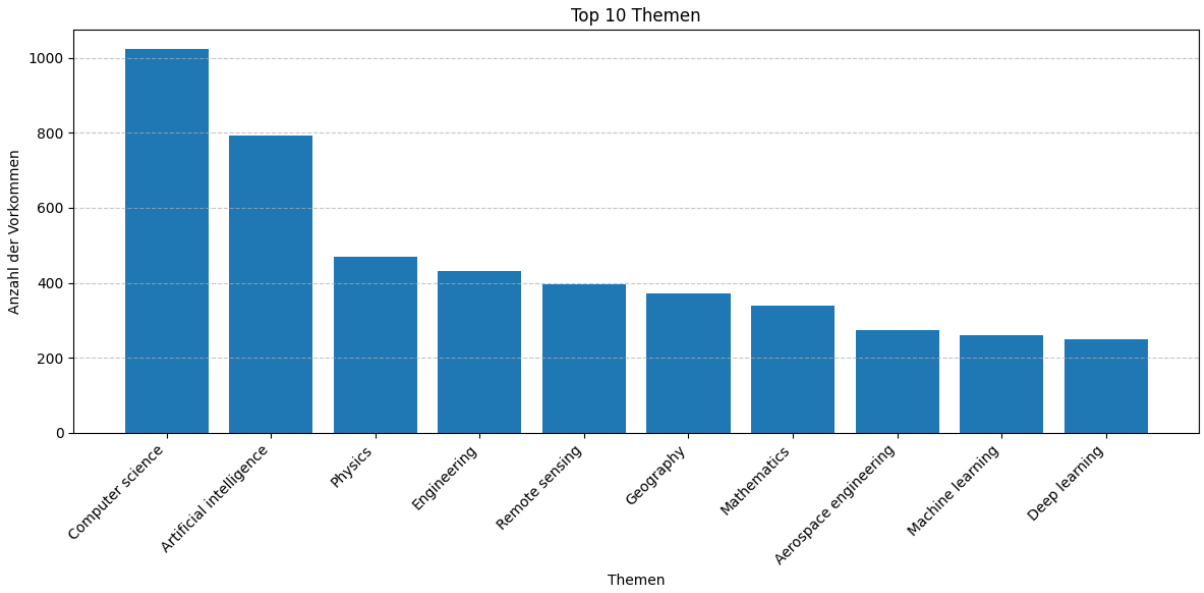
[4] S. Szott, „moodle.hdm-stuttgart.de,“ 26 07 2025. [Online]. Available: https://moodle.hdm-stuttgart.de/course/view.php?id=21617.

[5] OpenAI, „chatgpt.com,“ 26 07 2025. [Online]. Available: https://chatgpt.com/.

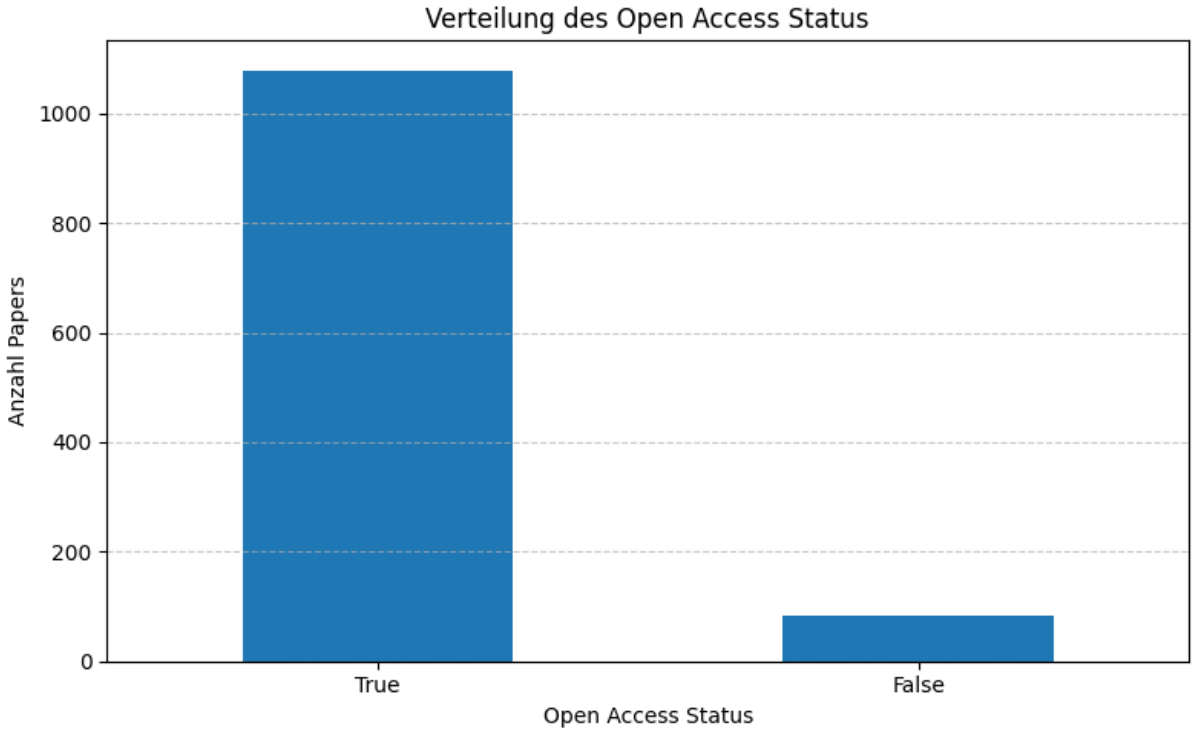
[6] Anthropic, „claude.ai,“ 26 07 2025. [Online]. Available: https://claude.ai.

Anhang

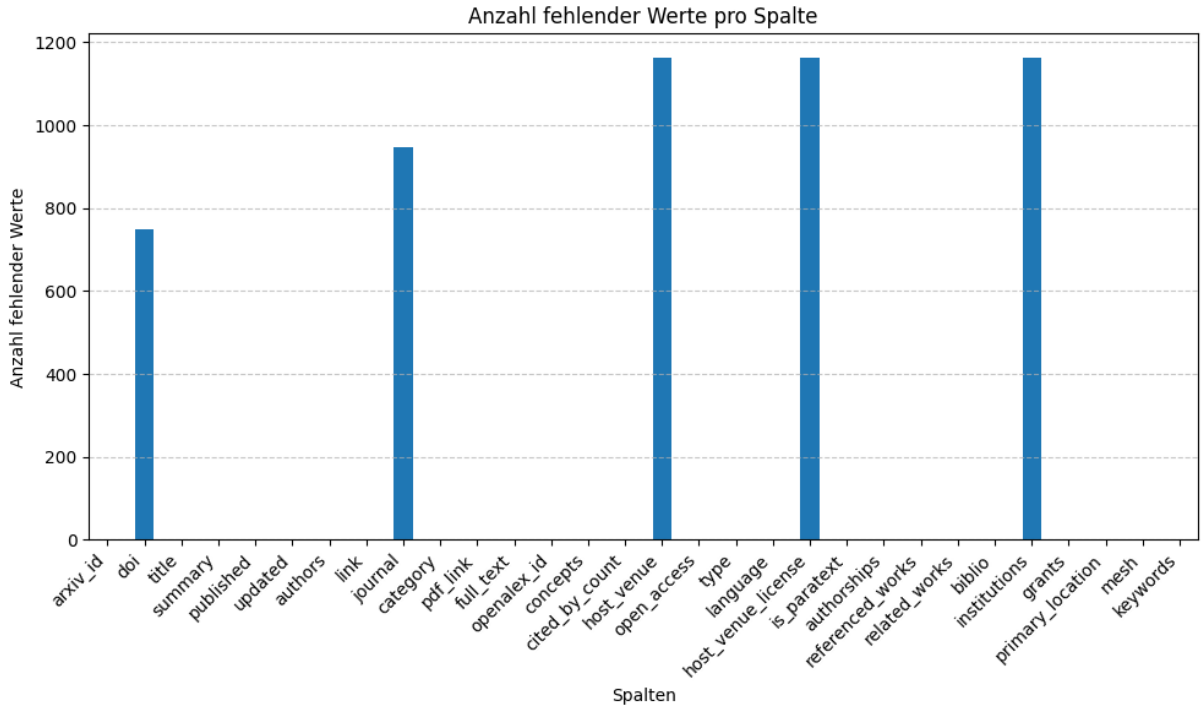
Anhang 1



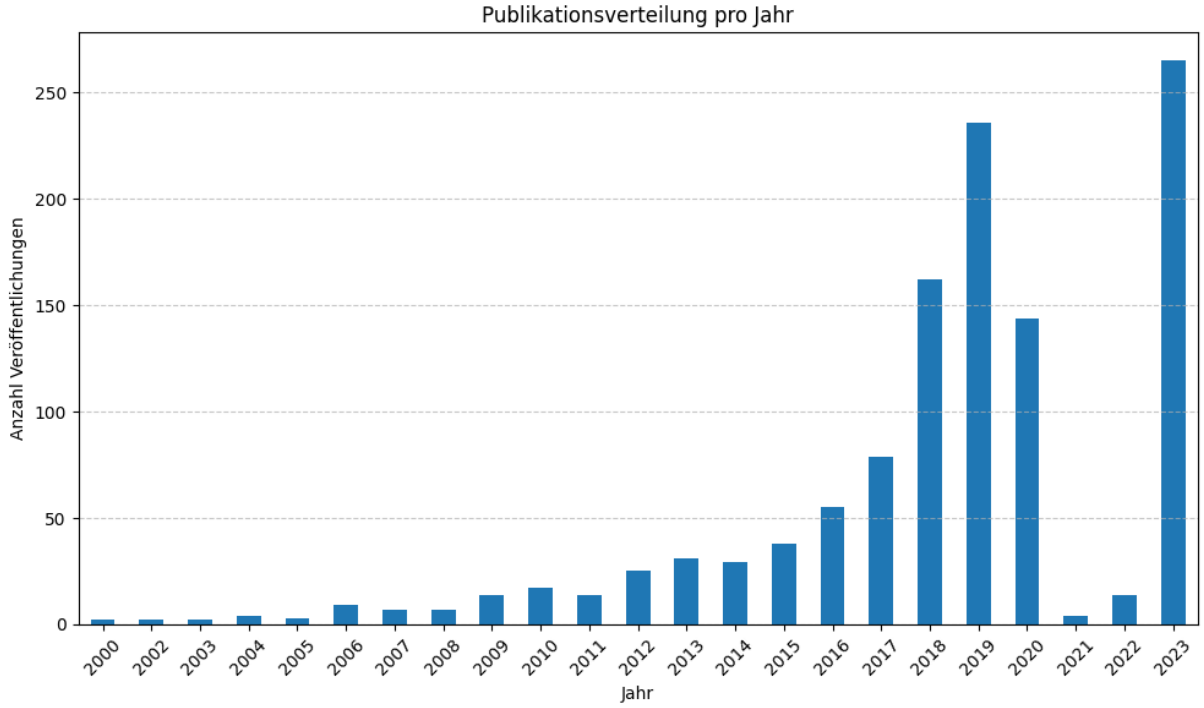
Anhang 2



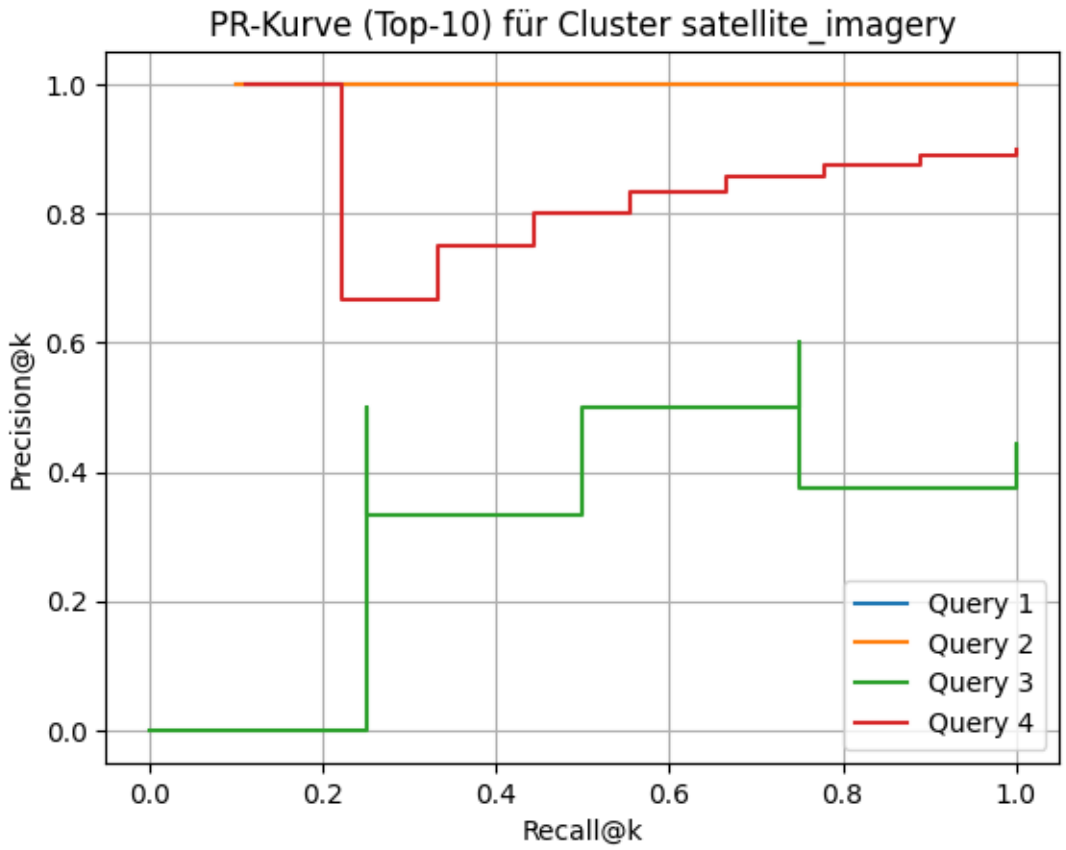
Anhang 3



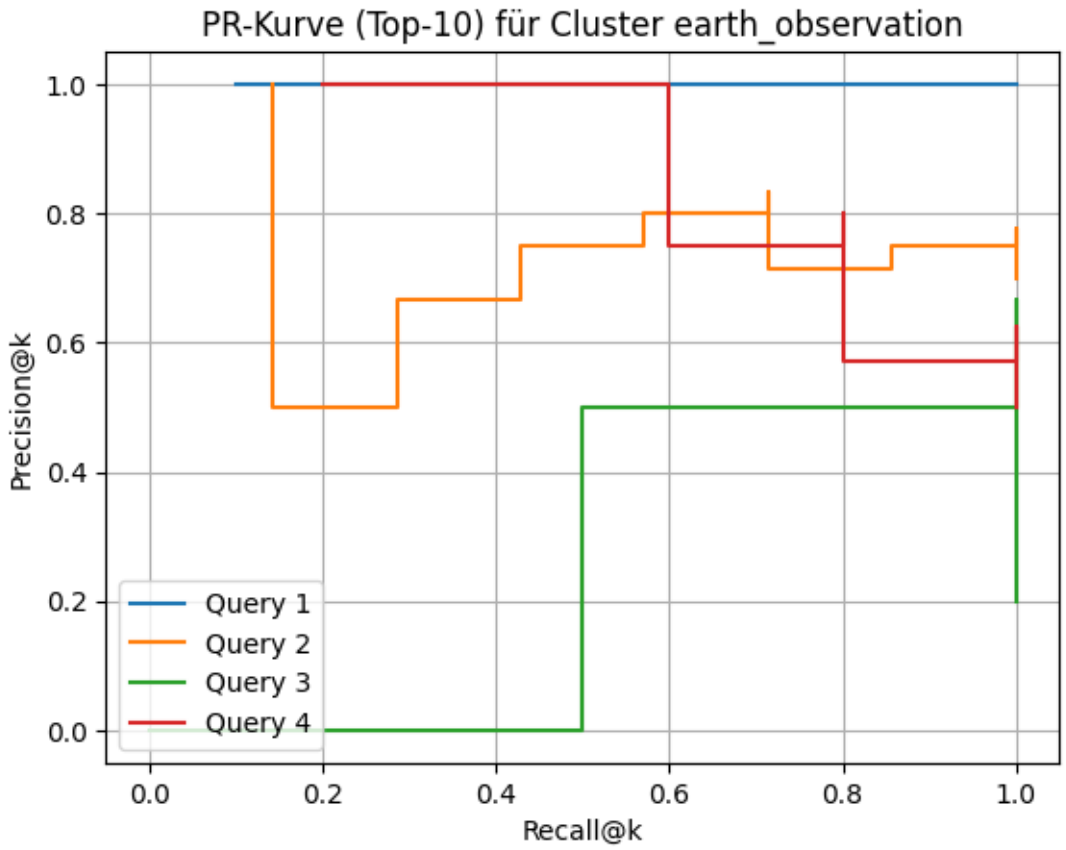
Anhang 4



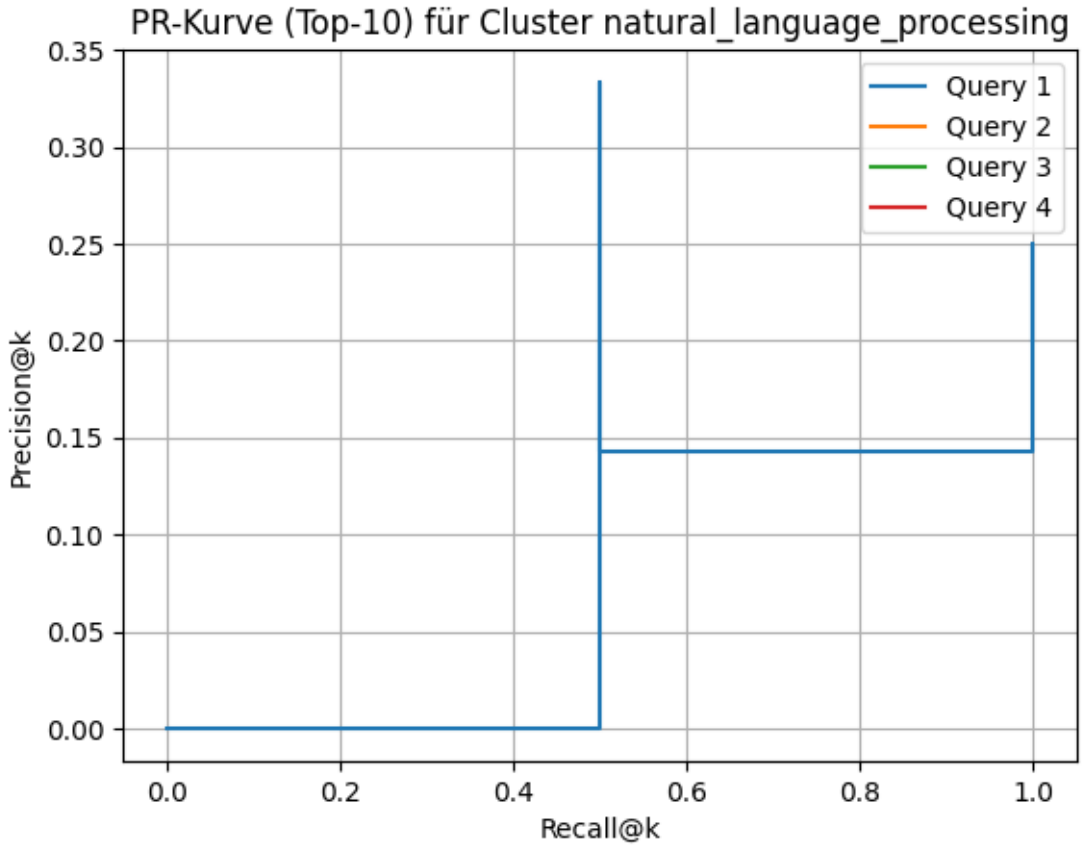
Anhang 5



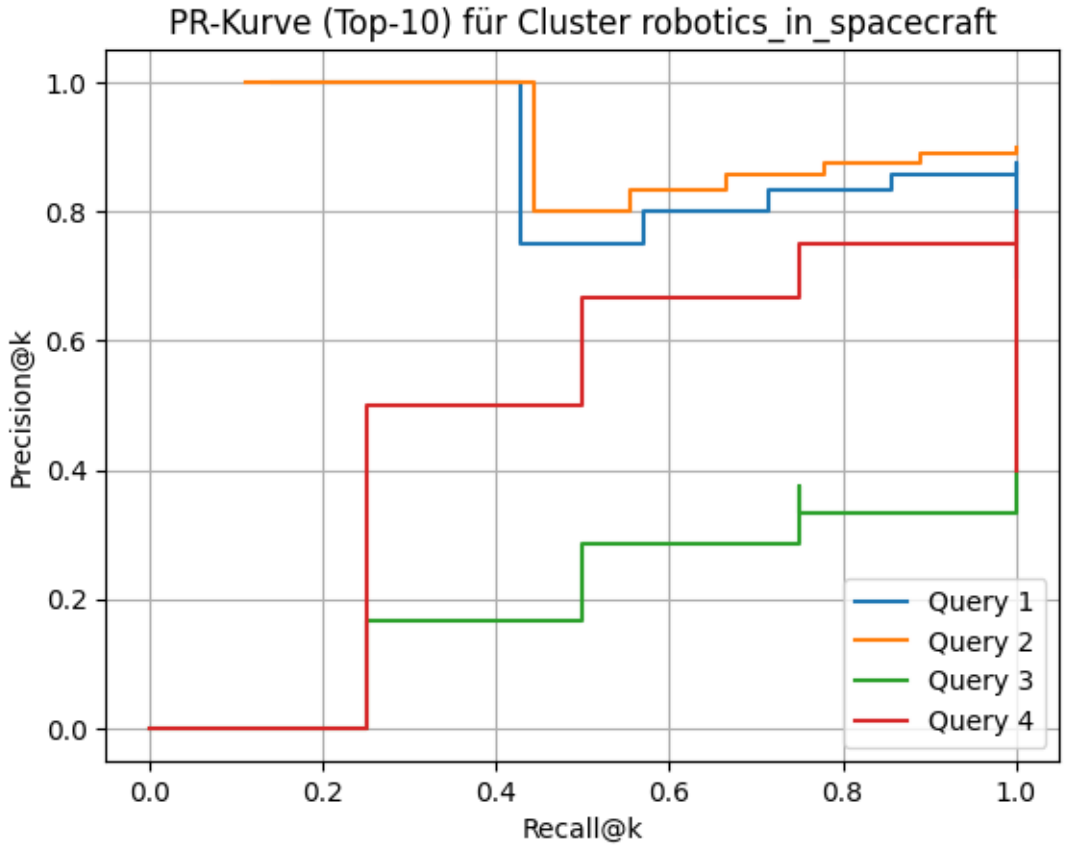
Anhang 6



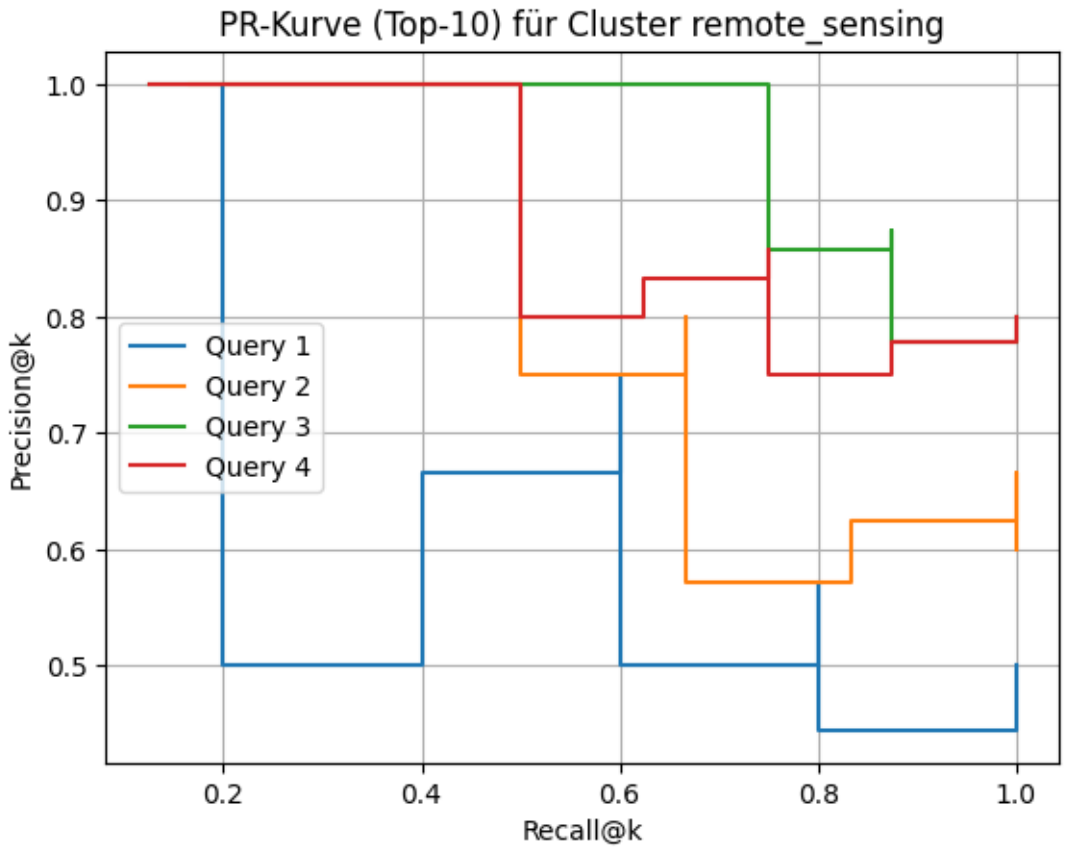
Anhang 7



Anhang 8



Anhang 9


PR-Kurve (Top-10) für Cluster remote_sensing
Precision@k
Recall@k
1.0
0.9
0.8
0.7
0.6
0.5
0.2
0.4
0.6
0.8
1.0
Query 1
Query 2
Query 3
Query 4